11-matchers-included-in-rspec-expectations/exercises/tokenizer_spec.rb
```
class Tokenizer
  def self.tokenize(string)
    string.split(/ +/)
  end
end
```

Run the spec, and read the failure message. Our spec caught the error, but it didn't give any context beyond just the six words we asked for. Moreover, if we ever update this spec, we have to take extra care to keep the length parameter, first(6), in sync with the list of expected words.

Change your spec to use a more future-proof matcher that doesn't require us to extract a hard-coded number of tokens.

Building Blocks of Nature

For this example, we'll be tearing apart the molecules making up our world around us. Fortunately, it's just a simulation. Create a new file with the following spec for water:

11-matchers-included-in-rspec-expectations/exercises/water_spec.rb
```
RSpec.describe Water do
  it 'is H2O' do
    expect(Water.elements.sort).to eq [:hydrogen, :hydrogen, :oxygen]
  end
end
```

At the top of your file, add an implementation of Water that's missing one of its hydrogen atoms:

11-matchers-included-in-rspec-expectations/exercises/water_spec.rb
```
class Water
  def self.elements
    [:oxygen, :hydrogen]
  end
end
```

Run your spec. It will fail correctly, but the output leaves something to be desired. We just get two collections dumped to the console, and it's up to us to read them by hand and find out what's different. With just a few items, comparing by hand is manageable, but differences become much harder to spot as the collections get bigger.

Also, take a look at that sort call we had to add. This spec has nothing to do with sorting, but we had to sort the collection to ensure we were just comparing the elements without regard to order.